Identifying Key Entities in Recipe Data

Custom NER model using a CRF | Abinas Patra

Summary: I trained a model that labels each word in a recipe as an ingredient, quantity or unit. On 84 recipes it had never seen, it got **98.3%** of words right. Units were the hardest label, and many mistakes come from inconsistent labels in the data.

1. The problem

Recipe websites and meal planning apps need to know what is in a recipe. Tagging words by hand is slow. A model can do this automatically, which turns free text into structured data. Users could then search recipes by the ingredients they have.

For example, in "3 tablespoon Gram flour", "3" is a quantity, "tablespoon" is a unit, and "Gram flour" is an ingredient. This task is called Named Entity Recognition (NER).

2. The data

The file has 285 recipes. Each has two columns: **input** (the ingredient words) and **pos** (one label for each word).

- **Missing values:** none, and every label is one of the 3 expected names.
- **Problem found:** rows 17, 27, 79, 164 and 207 each have one more word than labels. We cannot tell which label belongs to which word, so I removed them.
- **Clean data:** 280 recipes, split 70:30 into 196 training recipes and 84 validation recipes.

3. My approach

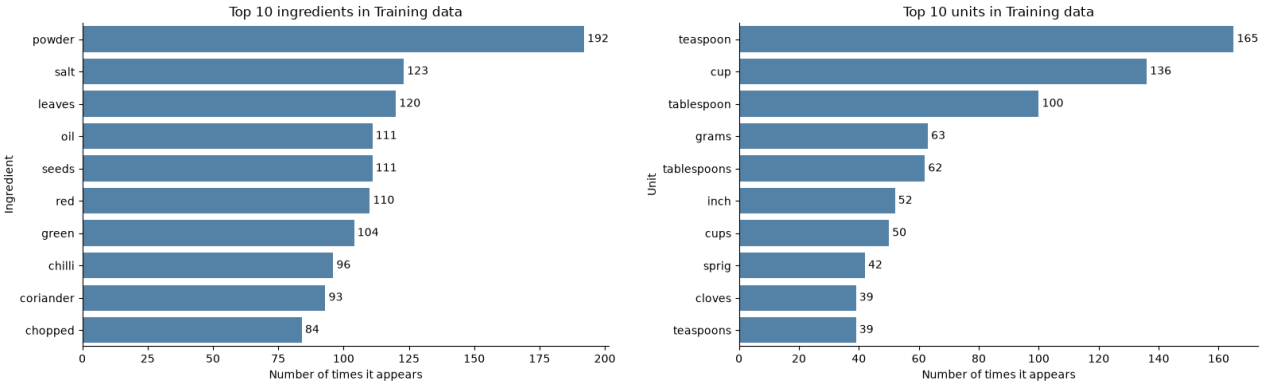
1. Loaded the data, split sentences into words and removed broken rows.
2. Split the recipes into training and validation sets.
3. Explored the training data with counts and bar charts.
4. Described each word with 32 clues (features), plus a class weight.
5. Trained a CRF model using the given settings.
6. Tested it on the validation set and studied its mistakes.

Tools used: pandas for tables, matplotlib and seaborn for charts, spaCy for grammar clues, and sklearn-crfsuite for the model. I chose a CRF because it looks at neighboring words, not only the word itself.

4. What the data shows

The training set has 7,114 words. Most of them are ingredients.

Label	Words	Share
ingredient	5,323	74.8%
quantity	980	13.8%
unit	811	11.4%



Top 10 ingredient words and unit words in the training data.

- **Labels are unbalanced.** A model could score about 75% just by always guessing "ingredient". So accuracy alone is not enough, and I also check each label.
- **Ingredient words are often parts of names.** "powder", "red" and "green" come from names like "red chilli". This is why neighboring words matter.
- **A few units dominate.** Teaspoon, tablespoon and cup, with their plurals, make up 68% of all units. A keyword list can catch most of them.
- **Spices are very common.** "coriander" and "chilli" are in the top 10, so the model may do less well on other types of food.

5. Clues for the model

A model cannot read words, so I described each word with clues:

Type	Examples	Why it helps
Basic	The word, its base form, word type, has a digit	Tells the model what the word looks like
Amount	Is it a fraction? Is it in my unit list?	Finds "1/2" or "cups"
Neighbor	Previous word, next word, first or last word	A word after "3" is often a unit

I also made keyword lists for units, quantities and cooking methods, plus a pattern that spots numbers like 2, 1.5, 1/2 and 1-1/2. Keywords are not perfect: "gram" is in my unit list, but "Gram flour" is a food.

6. Handling unbalanced labels

I gave rare labels bigger weights (total words divided by the label's count) and halved the ingredient weight: **unit 8.77, quantity 7.26, ingredient 0.67**.

An important choice: the weight clue must not use the true label, because that shows the model the answer. I tested this in a separate experiment:

Way of adding the weight clue	Validation accuracy	Is it real?
Uses the true label	99.9%	No, the model reads the answer
Same model on new recipes, with no labels available	73.3%	Yes, it finds almost no units or quantities
Uses a keyword guess (my choice)	**98.3%**	Yes

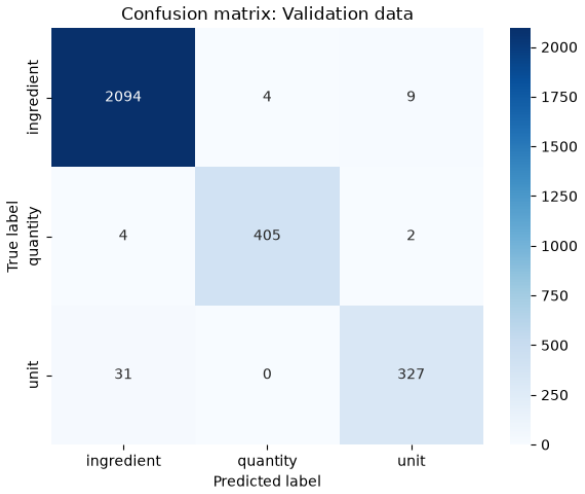
The weights added little. Without them, the same model scored 98.2%. The assignment also asks to set **class_weight** on the model. I set it, but the library's code never uses that name, so it does not change the learning.

7. Results

Label	Precision	Recall	F1	Words
ingredient	0.984	0.994	0.989	2,107
quantity	0.990	0.985	0.988	411
unit	0.967	0.913	0.940	358

Validation results. Precision: when the model says "unit", how often is it right? Recall: of all real units, how many did it find?

Accuracy was **99.2% on training** and **98.3% on validation**. The gap is small, so the model is not just memorizing. Unit recall is the weak spot: it fell from 97.8% in training to 91.3%.



Confusion matrix on the validation data.

How to read this: the diagonal boxes are correct answers.

Of the 50 mistakes, 31 are real units predicted as ingredients. That is the biggest single error.

8. Why the model makes mistakes

Pattern	Evidence	Reason
"Cloves"	11 of 50 mistakes	It can be a measure (2 cloves garlic) or a spice
Describing words	About 24 of 31 unit mistakes: "finely", "cut", "small"	The data sometimes labels these as units
Quantity words	"few" and "a", 3 mistakes each	They have no digits to spot
Rare label	Units cause 31 of 50 mistakes	The model has the fewest examples of them

Main insight: many mistakes come from inconsistent labels, not only from a weak model. For example, "chopped" is usually an ingredient in the training data, but once it is a unit.

9. Conclusion and next steps

The model works well and could help tag recipes automatically, but a person should still review its output, especially units. To improve it, I would:

- Fix the inconsistent labels first, since this looks like the biggest gain.
- Add more recipes, especially with rare units and other types of food.
- Add clues for words like "cloves" that depend on the next word.

10. Assumptions

- **Broken rows:** I removed the 5 rows instead of guessing their labels.
- **Split:** I used random_state=42, so the split stays the same on every run.
- **Ingredient penalty:** the task gave no number, so I chose 0.5.
- **Weight clue:** I used a keyword guess instead of the true label, as explained in section 6.
- **Extra clue:** I added is_cooking_method, for 32 clues in total.
- **Setup:** spaCy 3.5.3 needs an older setup, so I used Python 3.11, NumPy below version 2, and installed the English model directly.
- **Optional parts:** I did not do the optional validation data analysis in Section 5.